

TK 1: Sistem Persamaan Linear dan Least Square Problem

Kelompok A12 – Semester Gasal 2026/2027

Daftar Isi

Laporan 1: Analisis Perpindahan Penumpang Antarhalte	3
1 Pendahuluan dan Validasi Data	3
2 Formulasi Sistem dan Struktur Pita	4
3 Dua Solver dan Uji Ukuran Kecil	4
4 Waktu, Memori, dan Kompleksitas	5
5 Kondisi, Galat, dan Interpretasi	6
6 Kesimpulan dan Rekomendasi	7
 Laporan 2: Model SETAR untuk Prediksi Return Saham	 8
1 Pendahuluan	8
2 Formulasi matriks desain	8
3 Isu numerik dan mitigasi	9
4 Persamaan normal	9
5 QR dengan rotasi Givens	10
6 Perbandingan hasil dan biaya	12
7 Evaluasi luar sampel dan interpretasi	12
8 Pengaruh outlier	14
9 Kesimpulan	15
A Lampiran: Verifikasi dan Rincian Reproduksi	17

Laporan 1: Analisis Perpindahan Penumpang Antarhalte

Rangkuman

Enam matriks transisi kode B untuk $N = 16, 32, 64, 128, 256, 512$ dipakai untuk menghitung distribusi stasioner penumpang. Sistem asal singular; satu persamaan diganti dengan syarat skala dan solusinya dinormalkan. Kami membandingkan LU dense berpivot parsial dengan eliminasi pita berpivot parsial untuk $p = 2, q = 1$. Pada $N = 512$, solver pita memerlukan 20.09 ms dan 60 KiB, dibandingkan 164.22 ms dan 4108 KiB untuk dense. Kedua metode memberikan solusi dan residual yang sama pada ketelitian yang diamati. Kasus $N = 256$ hampir terpisah menjadi dua blok dan memiliki kondisi terbesar, $\kappa_1(B) \approx 1.71 \times 10^9$. Halte terakhir memiliki proporsi terbesar pada setiap ukuran. Untuk koridor besar, solver pita unggul dalam waktu dan memori, tetapi kondisi matriks tetap perlu dipantau.

1 Pendahuluan dan Validasi Data

Misalkan X_k adalah halte penumpang pada interval ke- k dan $T_{ij} = \Pr(X_{k+1} = j \mid X_k = i)$. Untuk setiap N , matriks T dibaca dari CSV kode B yang sesuai dengan kelompok genap A12. Validasi memeriksa bentuk $N \times N$, elemen hingga dan nonnegatif, jumlah tiap baris satu hingga toleransi 10^{-12} , serta keterhubungan kuat graf transisi. Dua pencarian graf, pada arah asli dan balik, memastikan rantai *irreducible*; elemen diagonal positif memastikan aperiodisitas [1]. Kondisi ini memberi distribusi stasioner tunggal dan konvergensi dari kondisi awal mana pun. Semua matriks lolos. Galat jumlah baris maksimum hanya 2.22×10^{-16} ; setiap baris mempunyai dua sampai empat elemen tak-nol pada offset $-1, 0, 1, 2$. Kasus $N = 256$ memiliki peluang lintas blok berorde 10^{-8} , sehingga valid namun hampir terpisah.

Tabel 1.1: Validasi data kode B. Semua ukuran lolos uji dimensi, nonnegativitas, jumlah baris, irreducibilitas, dan aperiodisitas.

N	Galat baris maks.	$\min T_{ii}$	$\min\{T_{ij} > 0 : i \neq j\}$	Valid
16	1.11×10^{-16}	0.835	0.01442	Ya
32	2.22×10^{-16}	0.835	0.01467	Ya
64	2.22×10^{-16}	0.835	0.01483	Ya
128	2.22×10^{-16}	0.835	0.01492	Ya
256	2.22×10^{-16}	0.835	1.50×10^{-8}	Ya
512	2.22×10^{-16}	0.835	0.01498	Ya

Tabel 1.1 memperlihatkan mengapa ukuran 256 perlu perhatian khusus: probabilitas perpindahan positif terkecil jauh lebih kecil daripada ukuran lain. Fungsi validasi juga diuji pada matriks kecil yang sengaja dibuat invalid, dan menolak nilai negatif, baris tidak stokastik, serta rantai yang tidak irreducible.

2 Formulasi Sistem dan Struktur Pita

Untuk vektor kolom proporsi $\pi^{(k)}$, hukum peluang total memberi $\pi^{(k+1)} = T^\top \pi^{(k)}$. Pada keadaan stasioner, $T^\top \pi = \pi$, $\mathbf{1}^\top \pi = 1$, dan $\pi_i \geq 0$. Dengan $A = I - T^\top$ diperoleh $A\pi = 0$. Karena $T\mathbf{1} = \mathbf{1}$, maka $\mathbf{1}^\top A = 0$: baris-baris A bergantung linear dan A singular. Baris pertama dapat diganti dengan syarat skala $z_1 = 1$:

$$B_{1,:} = e_1^\top, \quad B_{i,:} = A_{i,:} \ (i \geq 2), \quad b = e_1, \quad Bz = b, \quad \pi = \frac{z}{\mathbf{1}^\top z}. \quad (2.1)$$

Untuk rantai irreducible, setiap keadaan akhirnya dapat mencapai halte 1; submatriks transisi di luar halte 1 memiliki jari-jari spektral kurang dari satu. Karena itu B nonsingular [2]. Solusi z belum berjumlah satu, sehingga normalisasi terakhir dalam (2.1) diperlukan.

Lower bandwidth $p = \max_{B_{ij} \neq 0} (i - j)$ dan upper bandwidth $q = \max_{B_{ij} \neq 0} (j - i)$ dihitung otomatis dengan memindai indeks semua entri tak-nol dan memperbarui kedua maksimum. Diperoleh $p = 2, q = 1$ untuk semua ukuran. Transposisi T menukar offset $-1, 0, 1, 2$ menjadi $1, 0, -1, -2$; penggantian baris pertama dengan $e_1^\top$ tidak menambah elemen di luar pita. Jadi solver Thomas tridiagonal murni tidak berlaku. Penyimpanan dense membutuhkan N^2 bilangan, pita biasa $(p + q + 1)N = 4N$, dan pita dengan ruang *fill-in* untuk pivoting $(2p + q + 1)N = 6N$ bilangan [6, 4]. Pada $N = 512$ kebutuhan matriks dense 2 MiB, sedangkan array pita dengan *fill-in* hanya 24 KiB.

3 Dua Solver dan Uji Ukuran Kecil

Kedua solver menyelesaikan $Bz = b$ lalu menerapkan normalisasi (2.1). LU dense menyimpan faktor L dan U dalam satu array persegi. Pada langkah k , pivot dipilih sebagai indeks $r = \arg \max_{i \geq k} |B_{ik}|$, baris k, r ditukar, multiplier $\ell_{ik} = B_{ik}/B_{rk}$ dihitung, dan sisa baris diperbarui. Permutasi baris ikut diterapkan ke b sebelum substitusi maju dan balik, sehingga $PB = LU$ [3]. Solver pita memakai prinsip yang sama tetapi mencari pivot hanya pada baris $k, \dots, k + p$ serta memperbarui kolom sampai $k + p + q$. Faktor disimpan sebagai diagonal pada array berukuran $(2p + q + 1) \times N$; pertukaran dicatat sebagai vektor pivot. Ini merupakan perluasan algoritma Thomas untuk pita $p = 2, q = 1$ dengan pivoting.

Kode 3.1 Langkah utama LU dense dan eliminasi pita berpivot parsial

Masukan: B, b , serta bandwidth p, q untuk mode pita

- 1: pilih mode dense atau simpan diagonal B dalam array pita berukuran $(2p + q + 1) \times N$
 - 2: **for** $k = 1, \dots, N - 1$ **do**
 - 3: cari $r = \arg \max |B_{ik}|$ pada $i = k, \dots, N$ (dense) atau $i = k, \dots, \min(N, k + p)$ (pita)
 - 4: tukar baris aktif dan catat pivot; tolak sistem bila pivot nol
 - 5: **for** setiap baris aktif $i > k$ **do**
 - 6: $\ell_{ik} \leftarrow B_{ik}/B_{rk}$; perbarui entri setelah kolom k dalam jangkauan mode
 - 7: **end for**
 - 8: **end for**
 - 9: terapkan pertukaran pada b ; substitusi maju dan balik menghasilkan z
 - 10: **return** $z/(\mathbf{1}^\top z)$
-

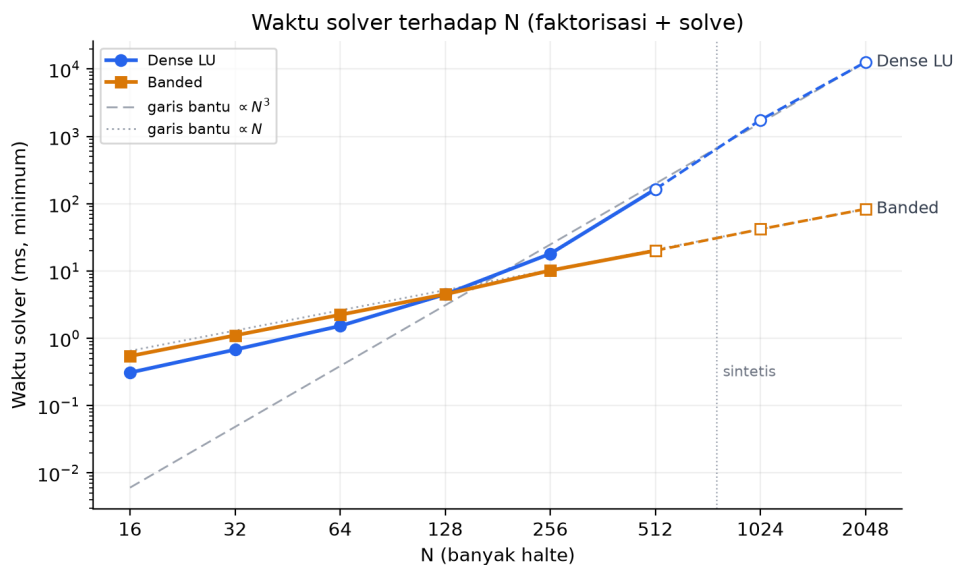
Kode 3.1 menunjukkan perbedaan jangkauan pivot dan pembaruan; algoritma linear utama diimplementasikan sendiri. Pada uji $B = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}$ dan $b = (3, 7, 19)^T$, kedua solver memberi $z = (1, -1, 2)^T$ dan $\max |PB - LU| = 0$. Uji rantai tiga halte dengan solusi $\pi = (1/4, 1/2, 1/4)^T$ juga tepat. Pada keenam data soal, tidak terjadi pertukaran pivot dan selisih maksimum antardistribusi dari kedua solver adalah nol pada mesin pengujian. Pemanggilan solver pustaka hanya dipakai sebagai verifikasi terpisah.

4 Waktu, Memori, dan Kompleksitas

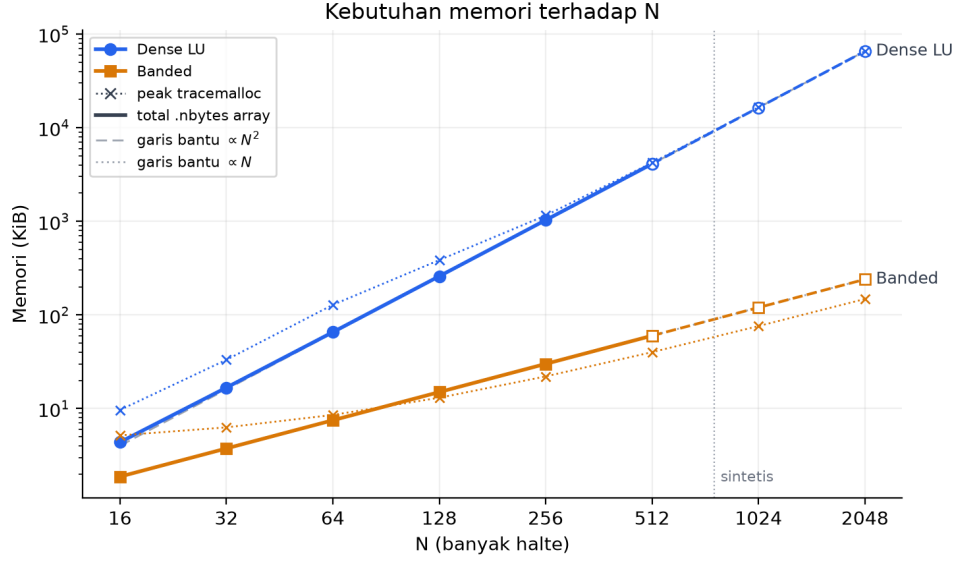
Pengukuran pada data soal dilakukan 15 kali per ukuran dan nilai minimum dilaporkan; waktu membaca CSV dipisahkan dari persiapan dan solver. Memori `.nbytes` mencakup array yang disimpan solver. Hasil pada Tabel 4.1 menunjukkan titik silang di sekitar $N = 128$: overhead perulangan Python membuat solver pita lebih lambat pada N kecil, tetapi pada $N = 512$ sekitar 8.2 kali lebih cepat. Gambar 4.1 dan 4.2 memperlihatkan tren waktu dan memori.

Tabel 4.1: Waktu solver minimum (ms) dan memori array solver (KiB) pada data soal.

N	Dense (ms)	Pita (ms)	Dense (KiB)	Pita (KiB)
16	0.310	0.544	4.4	1.9
32	0.678	1.104	16.8	3.8
64	1.527	2.235	65.5	7.5
128	4.500	4.512	259.0	15.0
256	18.041	10.179	1030.0	30.0
512	164.217	20.090	4108.0	60.0



Gambar 4.1: Waktu solver terhadap jumlah halte; titik 1024 dan 2048 berasal dari data sintetis.



Gambar 4.2: Memori array solver terhadap jumlah halte pada data soal dan data sintetis.

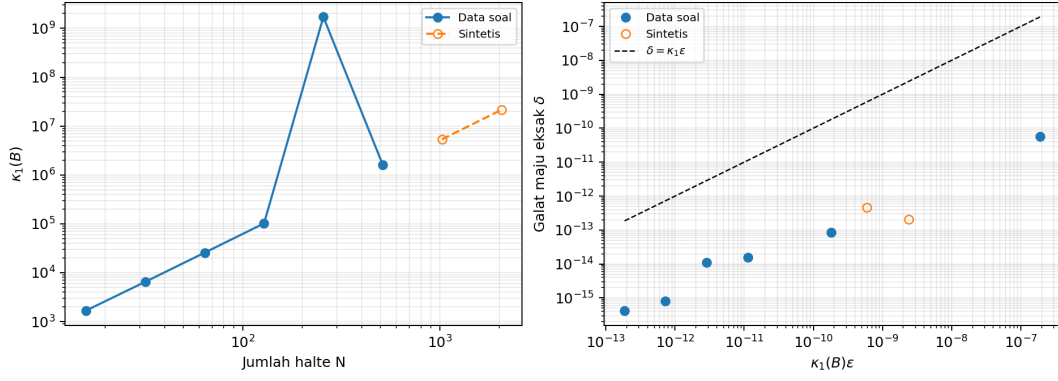
LU dense membutuhkan $O(N^3)$ operasi dan $O(N^2)$ ruang. Untuk pita dengan pivoting, jangkauan operasi per langkah dibatasi sekitar $p(p+q)$, sehingga waktunya $O(Np(p+q))$ dan ruangnya $O(N(2p+q+1))$ [6]. Pada $p=2, q=1$, keduanya linear terhadap N . Eksperimen sintetis $N=2048$ memberi 12.68 s untuk dense dan 83.28 ms untuk pita, sesuai arah pertumbuhan tersebut. Input CSV sendiri dense, sehingga pembacaan dan deteksi pita dapat berbiaya $O(N^2)$ meskipun solvernya linear; penyimpanan awal sebagai diagonal menghilangkan hambatan ini.

5 Kondisi, Galat, dan Interpretasi

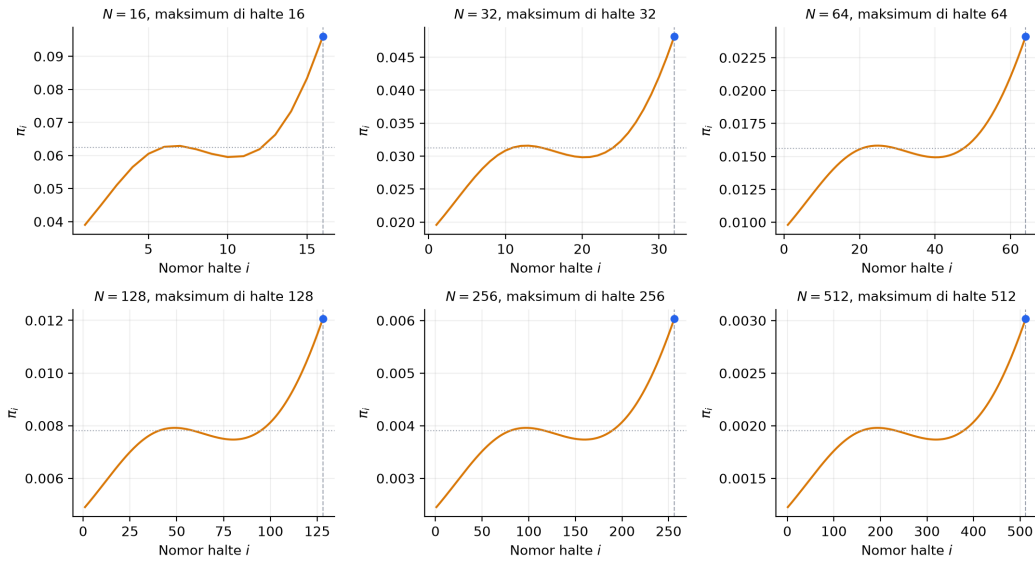
Seluruh eksperimen memakai $\kappa_1(B) = \|B\|_1 \|B^{-1}\|_1$ agar norma kondisi konsisten. Residual $r = \|T^T \pi - \pi\|_2$ dan galat normalisasi $e_{\text{norm}} = |\mathbf{1}^T \pi - 1|$ dihitung setelah penyelesaian. Tabel 5.1 menunjukkan kondisi sangat besar pada $N=256$ akibat penghubung antarblok yang amat kecil. Residual tetap kecil, tetapi residual kecil tidak menjamin galat maju kecil bila B buruk kondisinya [8]. Kedua solver memberi hasil identik pada data ini; galat maju terhadap acuan presisi tinggi pada $N=256$ sekitar 5.75×10^{-11} .

Tabel 5.1: Kondisi, residual, dan halte dengan proporsi maksimum; hasil kedua solver identik pada digit yang ditampilkan.

N	$\kappa_1(B)$	r	e_{norm}	Halte maks. (π_i)
16	1.68×10^3	1.55×10^{-17}	0	16 (0.09596)
32	6.52×10^3	1.30×10^{-17}	0	32 (0.04814)
64	2.57×10^4	9.66×10^{-18}	0	64 (0.02410)
128	1.02×10^5	5.95×10^{-18}	0	128 (0.01206)
256	1.71×10^9	3.85×10^{-18}	0	256 (0.00603)
512	1.62×10^6	3.21×10^{-18}	0	512 (0.00302)



Gambar 5.1: Bilangan kondisi terhadap N dan galat maju terhadap batas kondisi.



Gambar 5.2: Profil distribusi stasioner terhadap nomor halte pada enam ukuran data.

Gambar 5.1 menunjukkan lonjakan kondisi pada $N = 256$, sedangkan Gambar 5.2 memperlihatkan kenaikan π_i ke arah ujung koridor. Halte terakhir memiliki proporsi terbesar pada keenam ukuran; pada $N = 512$, $\pi_{512} \approx 0.00301659$, sedangkan $\pi_1 \approx 0.00122645$. Massa menumpuk di ujung karena perpindahan maju tidak diimbangi oleh halte setelah N . Ini menunjukkan kebutuhan armada dan petugas relatif lebih tinggi di bagian akhir koridor; interpretasi tetap terbatas pada model transisi yang diberikan.

6 Kesimpulan dan Rekomendasi

Kedua solver akurat pada enam matriks uji. Solver pita adalah pilihan utama untuk jumlah halte besar karena memori dan operasi tumbuh linear pada lebar pita tetap, sedangkan LU dense tumbuh kuadratik dalam ruang dan kubik dalam waktu. Pemeriksaan $\kappa_1(B)$ tetap diperlukan: rantai hampir terpisah pada $N = 256$ menunjukkan bahwa residual sangat kecil bisa menyertai sensitivitas solusi yang besar. Untuk data yang lebih buruk kondisinya, perbaikan iteratif atau algoritma GTH dapat dipertimbangkan [5, 8]. Rekomendasi tersebut berdasarkan kompleksitas, waktu, memori, dan galat yang terukur; kedua teknik tambahan belum diuji di sini.

Laporan 2: Model SETAR untuk Prediksi Return Saham

Rangkuman

Kami membangun model SETAR dua rezim dan dua lag dari 303 harga penutupan latih, menghasilkan sistem kuadrat terkecil 300×6 . Persamaan normal diselesaikan dengan eliminasi Gauss berpivot parsial dan metode kedua dengan QR rotasi Givens. Koefisien kedua metode berbeda 2.44×10^{-16} dalam norma dua; norma residual latih 0.15297705, RMSE latih 0.00883213, dan RMSE uji pada 103 return sambung 0.01196708. Bilangan kondisi $\kappa_2(A) = 192.61$ meningkat menjadi $\kappa_2(A^T A) = 37096.84$ pada persamaan normal. Givens lebih tahan terhadap pembulatan, meski akurasi empiris kedua metode sama pada data ini. Koefisien bullish menunjukkan momentum pada kedua lag, sedangkan koefisien bearish menunjukkan pembalikan; outlier dapat menggeser parameter dan residual.

1 Pendahuluan

Model SETAR dua rezim memprediksi return saham menggunakan dua return terdahulu dan ambang tanda return terakhir. Tujuan studi ini ialah membentuk sistem kuadrat terkecil yang bebas dari kebocoran target, menyelesaikannya dengan persamaan normal dan rotasi Givens sesuai aturan kelompok genap, lalu membandingkan stabilitas dan akurasi pada data latih serta data uji. Analisis juga mencakup biaya komputasi, sensitivitas outlier, interpretasi koefisien, dan grafik prediksi terhadap return aktual.

2 Formulasi matriks desain

Untuk harga penutupan P_t , return harian adalah $R_t = (P_t - P_{t-1})/P_{t-1}$. Baris pertama deret return tidak terdefinisi. Dengan ambang $c = 0$ dan dua lag, model untuk $t \geq 3$ dalam indeks array Python adalah

$$R_t = \begin{cases} \alpha_1 + \phi_{1,1}R_{t-1} + \phi_{1,2}R_{t-2} + \varepsilon_t, & R_{t-1} \geq 0, \\ \alpha_2 + \phi_{2,1}R_{t-1} + \phi_{2,2}R_{t-2} + \varepsilon_t, & R_{t-1} < 0. \end{cases} \quad (2.1)$$

Definisikan $x = (\alpha_1, \phi_{1,1}, \phi_{1,2}, \alpha_2, \phi_{2,1}, \phi_{2,2})^T$ dan $b_t = R_t$. Baris matriks desain adalah

$$A_t = \begin{cases} (1, R_{t-1}, R_{t-2}, 0, 0, 0), & R_{t-1} \geq 0, \\ (0, 0, 0, 1, R_{t-1}, R_{t-2}), & R_{t-1} < 0. \end{cases}$$

Persoalannya ialah $\min_x \|Ax - b\|_2$. Dari 303 harga latih diperoleh 302 return dan $m = 300$ observasi lengkap, sehingga $A \in \mathbb{R}^{300 \times 6}$, $x \in \mathbb{R}^6$, dan $b \in \mathbb{R}^{300}$. Ada 195 baris bullish dan 105 bearish.

Penentuan rezim dan lag selalu memakai return sebelum target. Ini mencegah kebocoran target ke dalam A . Pada data uji, return pertama dihitung dari harga pertama uji dan harga terakhir latih, sementara dua lag awal juga memakai riwayat latih. Karena itu seluruh 103 harga uji menjadi 103 target return yang sah; tidak ada parameter yang diestimasi ulang dari data uji.

3 Isu numerik dan mitigasi

Kolom intersep berisi 0 atau 1, sedangkan kolom return biasanya berorde 10^{-2} . Skala kolom yang berbeda serta korelasi antarlag dapat membuat A kurang baik kondisinya. Satu rezim yang sangat jarang muncul juga dapat membuat parameter rezim tersebut sukar diidentifikasi; kedua rezim pada data ini tetap mempunyai jauh lebih banyak observasi daripada tiga parameter lainnya. Return ekstrem dapat mendominasi jumlah kuadrat residual.

Mitigasi yang relevan ialah memeriksa rank dan bilangan kondisi, membangun lag tanpa memakai target, dan menggunakan transformasi ortogonal untuk menghindari kuadrat bilangan kondisi. Jika perlu, kolom prediktor dapat diskalakan sebelum penyelesaian dan koefisien dikonversi kembali ke satuan semula. Pensakalan harus dihitung dari data latih saja. Pada laporan ini koefisien dilaporkan dalam skala return asli.

4 Persamaan normal

Kondisi optimal kuadrat terkecil untuk matriks desain ber-rank penuh menghasilkan persamaan normal [7]. Syarat stasioner $\|Ax - b\|_2^2$ menghasilkan

$$A^T Ax = A^T b. \quad (4.1)$$

Matriks 6×6 dibentuk dari data latih, lalu sistemnya diselesaikan dengan eliminasi Gauss dan *partial pivoting* serta substitusi balik yang diimplementasikan sendiri. Dalam aritmetika eksak, $\kappa_2(A^T A) = \kappa_2(A)^2$ bila A ber-rank penuh. Untuk data ini, $\kappa_2(A) = 192.6054$ dan $\kappa_2(A^T A) = 37096.8440$. Membentuk persamaan normal karena itu berpotensi memperbesar pengaruh pembulatan [8, 7] dan kehilangan digit akurasi, walaupun hasil pada data ini masih sangat dekat dengan QR.

Kode 4.1 Penyelesaian Persamaan Normal via Eliminasi Gauss dengan *Partial Pivoting*

Masukan: $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ **Keluaran:** $x \in \mathbb{R}^n$ solusi least squares

```
1:  $M \leftarrow A^\top A$  ▷ Matriks normal  $n \times n$ 
2:  $d \leftarrow A^\top b$  ▷ Ruas kanan  $n \times 1$ 
   — Eliminasi maju dengan partial pivoting —
3: for  $k = 1$  to  $n$  do
4:    $p \leftarrow \arg \max_{k \leq i \leq n} |M_{ik}|$  ▷ Cari pivot terbesar
5:   if  $M_{pk} = 0$  then
6:     error “Matriks singular”
7:   end if
8:   Tukar baris  $M_k \leftrightarrow M_p$  dan  $d_k \leftrightarrow d_p$ 
9:   for  $i = k + 1$  to  $n$  do
10:     $f \leftarrow M_{ik}/M_{kk}$ 
11:    for  $j = k + 1$  to  $n$  do
12:       $M_{ij} \leftarrow M_{ij} - f \cdot M_{kj}$ 
13:    end for
14:     $d_i \leftarrow d_i - f \cdot d_k$ 
15:     $M_{ik} \leftarrow 0$ 
16:  end for
17: end for
   — Substitusi balik —
18: for  $i = n$  downto  $1$  do
19:    $x_i \leftarrow \frac{d_i - \sum_{j=i+1}^n M_{ij} x_j}{M_{ii}}$ 
20: end for
21: return  $x$ 
```

5 QR dengan rotasi Givens

Rotasi ortogonal menghindari pembentukan matriks normal sehingga lebih tahan terhadap galat pembulatan [6, 7, 8]. Untuk menghilangkan entri a_{ji} di bawah diagonal, ambil $u = a_{ii}$, $v = a_{ji}$, $\rho = \sqrt{u^2 + v^2}$, $c = u/\rho$, dan $s = v/\rho$. Matriks G adalah identitas kecuali blok pada baris dan kolom i, j :

$$G_{\{i,j\},\{i,j\}} = \begin{pmatrix} c & s \\ -s & c \end{pmatrix}, \quad G \begin{pmatrix} u \\ v \end{pmatrix} = \begin{pmatrix} \rho \\ 0 \end{pmatrix}. \quad (5.1)$$

Penerapan seluruh rotasi memberi $R = G_k \cdots G_1 A$ dan $Q_{\text{QR}} = (G_k \cdots G_1)^\top$, sehingga $A = Q_{\text{QR}} R$. Jika variabel program Q menyimpan $G_k \cdots G_1$, ruas kanan yang benar adalah $d = Q @ b$; lalu selesaikan enam baris pertama $Rx = d$ dengan substitusi balik. Ini adalah orientasi yang dipakai pada perhitungan.

Kode 5.1 Penyelesaian Least Squares via QR Rotasi Givens

Masukan: $A \in \mathbb{R}^{m \times n}$ ($m > n$), $b \in \mathbb{R}^m$

Keluaran: $x \in \mathbb{R}^n$ solusi least squares

```
1:  $R \leftarrow A$  (salinan),  $d \leftarrow b$  (salinan)
   — Triangularisasi  $R$  dengan rotasi Givens —
2: for  $i = 1$  to  $n$  do                                     ▷ Iterasi tiap kolom
3:   for  $j = i + 1$  to  $m$  do                                   ▷ Eliminasi elemen subdiagonal
4:     if  $R_{ji} = 0$  then
5:       continue                                             ▷ Sudah nol, lewati
6:     end if
7:      $\rho \leftarrow \sqrt{R_{ii}^2 + R_{ji}^2}$ 
8:      $c \leftarrow R_{ii}/\rho$ ,  $s \leftarrow R_{ji}/\rho$ 
       — Terapkan rotasi ke baris  $i$  dan  $j$  —
9:     for  $k = i$  to  $n$  do                                     ▷ Perbarui kolom  $R$ 
10:       $(R_{ik}, R_{jk}) \leftarrow (c \cdot R_{ik} + s \cdot R_{jk}, -s \cdot R_{ik} + c \cdot R_{jk})$ 
11:    end for
12:     $(d_i, d_j) \leftarrow (c \cdot d_i + s \cdot d_j, -s \cdot d_i + c \cdot d_j)$        ▷ Perbarui ruas kanan
13:     $R_{ji} \leftarrow 0$ 
14:  end for
15: end for
   — Substitusi balik pada  $R_{1:n, 1:n} x = d_{1:n}$  —
16: for  $i = n$  downto  $1$  do
17:    $x_i \leftarrow \frac{d_i - \sum_{k=i+1}^n R_{ik} x_k}{R_{ii}}$ 
18: end for
19: return  $x$ 
```

Verifikasi rotasi pertama

Dua baris pertama matriks desain adalah

$$\begin{pmatrix} 0 & 0 & 0 & 1 & -0.00831972 & 0.00243800 \\ 1 & 0.00737754 & -0.00831972 & 0 & 0 & 0 \end{pmatrix}.$$

Entri subdiagonal pertama adalah $a_{21} = 1$, sedangkan $a_{11} = 0$. Maka $c = 0$, $s = 1$, dan blok rotasi pertama adalah $\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$. Dua baris pertama $G_1 A$ menjadi

$$\begin{pmatrix} 1 & 0.00737754 & -0.00831972 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 & 0.00831972 & -0.00243800 \end{pmatrix}.$$

Dengan demikian entri $(2, 1)$ tepat menjadi nol. Seluruh rotasi berikutnya memakai rumus yang sama.

6 Perbandingan hasil dan biaya

Tabel 6.1: Perbandingan pada matriks desain yang sama. Biaya Givens berlaku untuk rotasi dua baris langsung, sedangkan implementasi yang membentuk Q eksplisit memerlukan biaya tambahan; RMSE uji memakai 103 return sambung.

Ukuran	Persamaan normal	QR Givens
$\ Ax - b\ _2$	0.15297705	0.15297705
RMSE latih	0.00883213	0.00883213
RMSE uji	0.01196708	0.01196708
Norma selisih koefisien	2.44×10^{-16}	
FLOP teoretis utama	$2mn^2 + 2mn + 2n^3/3$	$O(mn^2)$
Memori tambahan	$O(n^2)$	$O(m^2)$ jika Q eksplisit

Tabel 6.2: Koefisien SETAR hasil kedua metode; selisihnya hanya pada digit pembulatan.

Parameter	Persamaan normal	QR Givens
α_1	0.00151337	0.00151337
$\phi_{1,1}$	0.17215290	0.17215290
$\phi_{1,2}$	0.16607285	0.16607285
α_2	-0.00121269	-0.00121269
$\phi_{2,1}$	-0.36037682	-0.36037682
$\phi_{2,2}$	-0.10980804	-0.10980804

Untuk $m \times n$ dengan $m \gg n$, pembentukan $A^T A$ memerlukan sekitar $2mn^2$ FLOP, $A^T b$ sekitar $2mn$, dan eliminasi $n \times n$ sekitar $2n^3/3$. Untuk data ini ($m = 300, n = 6$), total FLOP persamaan normal sekitar $2(300)(36) + 2(300)(6) + \frac{2}{3}(216) \approx 25\,344$. Givens memperbarui hanya dua baris aktif per rotasi; ada paling banyak 1779 rotasi sehingga batas atas FLOP sekitar 83 ribu. Kedua metode berorde $O(mn^2)$ untuk matriks tinggi, dengan konstanta Givens lebih besar sekitar $3,3\times$.

Jika A disimpan dense, keduanya memakai $O(mn)$ memori dasar. Persamaan normal menambah $O(n^2)$; A berisi 1800 bilangan `float64` (14.4 kB) dan $A^T A$ hanya 36 bilangan (288 byte). Implementasi notebook menyimpan Q berukuran 300×300 dan membuat G berukuran sama, masing-masing sekitar 720 kB; perkalian dense penuh untuk setiap rotasi juga mahal. Pembaruan langsung dua baris $[A \mid b]$, seperti pada skrip reproduksi laporan, menghindari penyimpanan Q dan menghasilkan solusi yang sama. Keuntungan utama QR ialah stabilitas terhadap pembulatan, bukan jaminan residual yang lebih kecil pada setiap data. Residual dan koefisien yang sama pada kasus ini konsisten dengan kondisi matriks yang belum cukup ekstrem untuk memisahkan hasil kedua metode dalam presisi ganda.

7 Evaluasi luar sampel dan interpretasi

RMSE dihitung sebagai $\sqrt{\sum_{t=1}^N (R_t - \hat{R}_t)^2 / N}$. Hasil uji 0.01196708 lebih besar daripada hasil latih 0.00883213, sehingga kesalahan prediksi meningkat di luar sampel.

Karena parameter kedua metode praktis identik (selisih $\sim 10^{-16}$), akurasi prediksi dan residualnya juga identik pada presisi yang dilaporkan. Dari segi **akurasi empiris**, tidak ada perbedaan: kedua metode menghasilkan RMSE latih dan uji yang sama persis. Dari segi **stabilitas numerik**, QR Givens lebih unggul karena bekerja langsung pada A dengan $\kappa_2(A) = 192.61$, sedangkan persamaan normal bekerja pada $A^T A$ yang bilangan kondisinya meningkat kuadratik menjadi 37 096.84. Pada data ini selisih tersebut belum memisahkan hasil karena presisi ganda masih memadai, tetapi QR akan lebih tahan terhadap pembulatan bila skala prediktor atau korelasi antarlag memburuk. **Kesimpulan:** kedua metode setara dalam akurasi; QR Givens lebih direkomendasikan untuk stabilitas numerik.

Persamaan estimasi akhirnya adalah

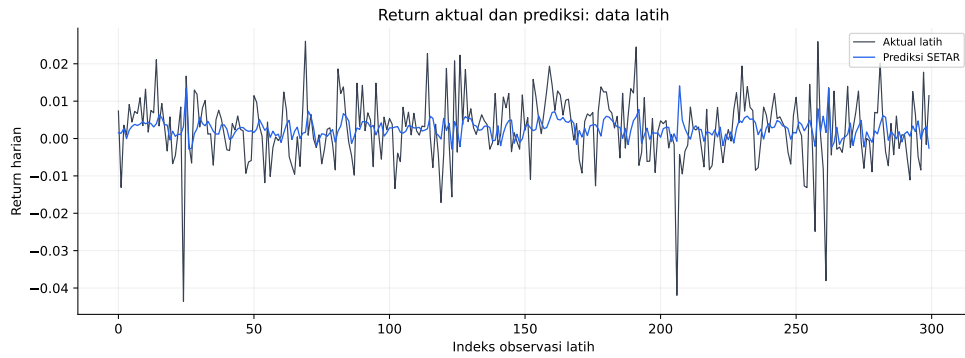
$$\hat{R}_t = \begin{cases} 0.00151337 + 0.17215290R_{t-1} + 0.16607285R_{t-2}, & R_{t-1} \geq 0, \\ -0.00121269 - 0.36037682R_{t-1} - 0.10980804R_{t-2}, & R_{t-1} < 0. \end{cases} \quad (7.1)$$

Kedua koefisien lag pada rezim bullish positif, sedangkan kedua koefisien lag pada rezim bearish negatif. Secara rinci:

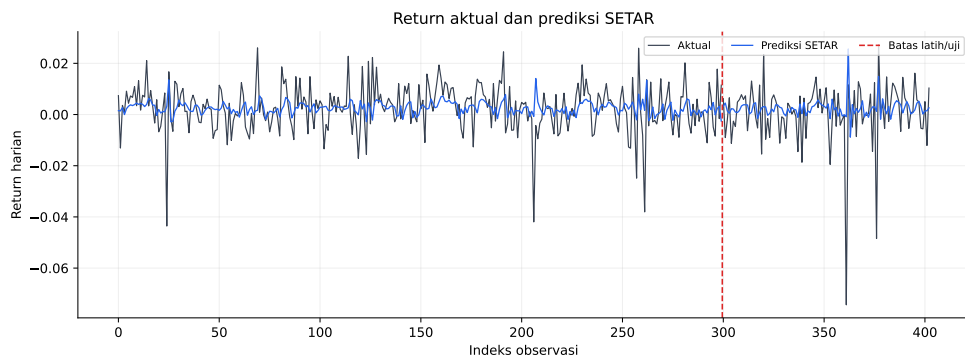
- $\alpha_1 = 0.00151 > 0$: *drift* positif pada rezim bullish. Tanpa pengaruh lag, model memperkirakan kenaikan harian baseline sekitar 0.15%.
- $\phi_{1,1} = 0.172 > 0$: koefisien lag-1 bullish positif, menunjukkan **momentum persistence**—return positif kemarin cenderung dilanjutkan hari ini.
- $\phi_{1,2} = 0.166 > 0$: koefisien lag-2 bullish juga positif, memperkuat tren jangka pendek dua hari. Pasar bullish memiliki memori positif.
- $\alpha_2 = -0.00121 < 0$: *drift* negatif pada rezim bearish. Tanpa pengaruh lag, ada tekanan turun baseline sekitar -0.12% per hari.
- $\phi_{2,1} = -0.360 < 0$: koefisien lag-1 bearish negatif dengan magnitudo paling besar, menunjukkan **mean-reversion** yang kuat—penurunan kemarin cenderung berbalik naik hari ini (*technical rebound*).
- $\phi_{2,2} = -0.110 < 0$: koefisien lag-2 bearish juga negatif, menyatakan pengaruh berlawanan arah terhadap return dua hari sebelumnya; arah kontribusi aktual bergantung pada tanda lag tersebut.

Interpretasi di atas berlaku dalam kerangka model, bukan bukti hubungan sebab-akibat ataupun jaminan kinerja investasi.

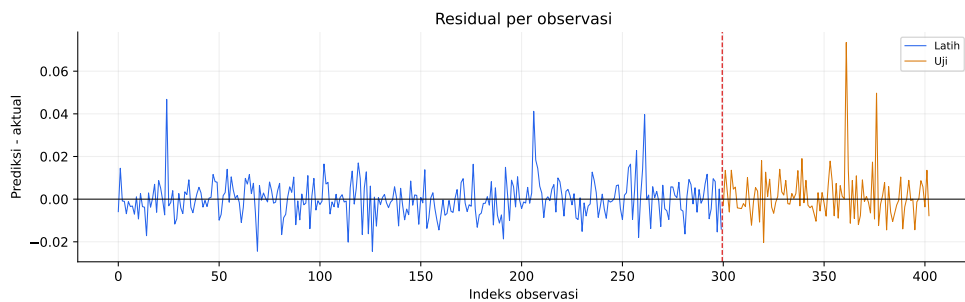
Gambar 7.1 memperlihatkan return aktual dan prediksi pada data latih. Gambar 7.2 menunjukkan segmen uji sebagai kelanjutan kronologis dengan batas yang jelas.



Gambar 7.1: Return aktual dan prediksi SETAR pada 300 observasi latih.



Gambar 7.2: Return aktual dan prediksi dari model yang dilatih pada segmen kiri. Garis putus-putus menunjukkan batas latih/uji; indeks uji pertama memakai riwayat harga latih.



Gambar 7.3: Residual prediksi per observasi. Beberapa deviasi besar tetap muncul walaupun parameter diperoleh dengan meminimalkan jumlah kuadrat galat latih.

8 Pengaruh outlier

Gambar 7.3 memperlihatkan galat per observasi, sedangkan Tabel 8.1 mengukur perubahan setelah observasi ekstrem dihapus. Fungsi objektif $\sum_t (R_t - \hat{R}_t)^2$ memberi bobot kuadrat pada tiap galat. Karena itu kejutan harga ekstrem dapat menaikkan norma residual secara tajam dan menarik koefisien ke arah observasi tersebut. Pada data latih, return

absolut terbesar sekitar 0.04357 dan galat prediksi absolut terbesar sekitar 0.04681 (29 Januari 2024). Galat maksimum pada data uji sekitar 0.07347.

Untuk menguji sensitivitas, kami mengestimasi ulang koefisien setelah menghapus satu dan tiga observasi dengan residual terbesar dari data latih, lalu mengevaluasi pada seluruh data latih dan uji yang sama.

Tabel 8.1: Sensitivitas terhadap outlier. Koefisien diestimasi ulang tanpa observasi berresidual terbesar.

Metrik	Data penuh	Hapus 1 outlier	Hapus 3 outlier
RMSE latih	0.00883213	0.00883439	0.00884748
RMSE uji	0.01196708	0.01200241	0.01203210
$\ x_{\text{penuh}} - x_{\text{baru}}\ _2$	—	7.62×10^{-3}	3.03×10^{-2}

Penghapusan satu outlier (29 Januari 2024) menggeser koefisien rezim bullish—terutama $\phi_{1,1}$ berubah sebesar +0.0067—sementara koefisien rezim bearish tidak berubah karena outlier tersebut berada di rezim bullish. RMSE latih dan uji sedikit meningkat karena koefisien baru tidak lagi mengakomodasi observasi ekstrem tersebut. Penghapusan tiga outlier memperbesar pergeseran koefisien empat kali lipat. Hasil ini menunjukkan bahwa satu observasi saja sudah mampu memengaruhi koefisien secara terukur, konsisten dengan sensitivitas kuadrat terkecil terhadap observasi ekstrem.

Kedua solver berhasil menyelesaikan model yang sama dan menghasilkan angka yang identik pada ketelitian pelaporan. Untuk implementasi yang efisien dan lebih tahan terhadap pembulatan, QR Givens sebaiknya diterapkan langsung pada dua baris matriks kerja dan ruas kanan, tanpa membentuk Q atau G dense penuh. Persamaan normal masih layak sebagai pembanding pada data ini, tetapi bilangan kondisinya yang kuadrat perlu diperhatikan jika skala atau korelasi prediktor meningkat.

9 Kesimpulan

Persamaan normal dan Givens menghasilkan solusi yang sama sampai presisi yang dilaporkan: norma residual latih 0.15297705, RMSE latih 0.00883213, dan RMSE uji 0.01196708. Pembentukan $A^T A$ meningkatkan bilangan kondisi dari 192.61 menjadi 37096.84; rotasi Givens karena itu lebih aman terhadap pembulatan bila data menjadi lebih buruk kondisinya. Untuk biaya yang wajar, rotasi diterapkan langsung ke dua baris matriks dan ruas kanan tanpa menyimpan Q eksplisit. Koefisien bullish menunjukkan momentum pada kedua lag, sedangkan koefisien bearish menunjukkan pembalikan dalam model ini; akurasi luar sampel lebih rendah daripada akurasi latih.

Reproduksibilitas. Seluruh langkah komputasi telah dijelaskan dalam dua laporan ini. Program yang digunakan adalah `soal1/soal1.ipynb` dan `soal2/program.ipynb`; skrip `soal2/report/analyze.py` mengulang perhitungan dan grafik SETAR. Ketiganya beserta CSV input dan README disertakan dalam ZIP submisi.

Referensi

- [1] J. R. Norris, *Markov Chains*. Cambridge University Press, 1997, Bab 1.3 dan 1.7–1.8.
- [2] W. J. Stewart, *Introduction to the Numerical Solution of Markov Chains*. Princeton University Press, 1994.
- [3] M. T. Heath, *Scientific Computing: An Introductory Survey*, edisi ke-2. McGraw-Hill, 2002, Bab 1–2.
- [4] E. Anderson dkk., *LAPACK Users' Guide*, edisi ke-3. SIAM, 1999.
- [5] W. K. Grassmann, M. I. Taksar, dan D. P. Heyman, “Regenerative analysis and steady state distributions for Markov chains,” *Operations Research*, vol. 33, no. 5, hlm. 1107–1116, 1985.
- [6] G. H. Golub dan C. F. Van Loan, *Matrix Computations*, edisi ke-4. Johns Hopkins University Press, 2013, Bab 5 dan Subbab 5.1.8 (Givens rotations).
- [7] L. N. Trefethen dan D. Bau III, *Numerical Linear Algebra*. SIAM, 1997, Kuliah 11 (least squares), 18 (conditioning of least squares), dan 19 (stability of least squares algorithms).
- [8] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, edisi ke-2. SIAM, 2002, Bab 7 dan 20.

A Lampiran: Verifikasi dan Rincian Reproduksi

A.1 Uji solver untuk Soal 1

Sebagai uji terukur bagi faktor LU dan permutasi, digunakan

$$B = \begin{pmatrix} 2 & 1 & 1 \\ 4 & 3 & 3 \\ 8 & 7 & 9 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 7 \\ 19 \end{pmatrix}.$$

Baris ketiga dipilih sebagai pivot pertama. Dengan urutan baris akhir $(3, 1, 2)$, kedua implementasi menghasilkan

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 1/4 & 1 & 0 \\ 1/2 & 2/3 & 1 \end{pmatrix}, \quad U = \begin{pmatrix} 8 & 7 & 9 \\ 0 & -3/4 & -5/4 \\ 0 & 0 & -2/3 \end{pmatrix}.$$

Perkalian langsung memberi $PB = LU$ dan penyelesaian menghasilkan $z = (1, -1, 2)^\top$. Pada uji terpisah, matriks pita 8×8 dengan $p = 2, q = 1$ dan diagonal yang diperkecil 100 kali memicu tujuh pertukaran baris serta delapan elemen *fill-in*; faktor U melebar hingga upper bandwidth $p + q = 3$. Ini memeriksa jalur pivot yang tidak terpakai oleh keenam data soal. Pada rantai tiga halte dengan distribusi acuan $(1/4, 1/2, 1/4)^\top$, kedua solver mengembalikan distribusi tersebut. Uji matriks singular $(\frac{1}{2} \frac{2}{4})$ ditolak karena pivot akhirnya nol.

A.2 Nilai rinci dan rotasi pertama untuk Soal 2

Ada 303 harga latih yang menghasilkan 302 return; setelah dua lag, matriks desain mempunyai 300 baris. Segmen uji mempunyai 103 harga dan 103 return karena return pertama memakai harga penutupan terakhir dari segmen latih. Untuk dua baris pertama desain, $a_{11} = 0$ dan $a_{21} = 1$, sehingga rotasi pertama mempunyai $c = 0$, $s = 1$ dan blok $\begin{pmatrix} 0 & 1 \\ -1 & 0 \end{pmatrix}$. Setelah G_1 diterapkan, entri $(2, 1)$ tepat nol; baris lain tidak berubah pada langkah ini.

Tabel A.1: Koefisien SETAR dari QR Givens sebelum pembulatan laporan.

Parameter	Nilai
α_1	0.001513374224392619
$\phi_{1,1}$	0.17215289727492178
$\phi_{1,2}$	0.16607285095297825
α_2	-0.0012126885538670722
$\phi_{2,1}$	-0.3603768239374077
$\phi_{2,2}$	-0.10980803854712604

Tabel A.1 memuat nilai yang digunakan untuk prediksi tanpa membulatkan koefisien terlebih dahulu. Selisih norma dua antara koefisien persamaan normal dan Givens ialah 2.43949×10^{-16} . ZIP submisi menyertakan kedua notebook, delapan CSV input, keluaran eksperimen, dan README dengan langkah menjalankan ulang program.

A.3 Kode inti dari implementasi

Kode A.1 menampilkan bagian faktorisasi kedua solver pada Soal 1. Substitusi segitiga dan normalisasi telah dijabarkan pada Kode 3.1; implementasi lengkap disertakan dalam notebook. Kode A.2 menampilkan solver persamaan normal dan Givens dari skrip reproduksi Soal 2. Rotasi diterapkan langsung pada dua baris matriks kerja dan ruas kanan, lalu diakhiri substitusi balik.

Kode A.1: Faktorisasi LU dense dan pita pada Soal 1.

```
# Extracted verbatim from soal1/soal1.ipynb
import numpy as np

def lu_dense(B: np.ndarray) -> tuple[np.ndarray, np.ndarray]:
    LU = B.astype(float).copy()
    N = LU.shape[0]
    perm = np.arange(N)
    for k in range(N - 1):
        # pivot: elemen dengan nilai mutlak terbesar di kolom k, baris k..N-1
        r = k + int(np.argmax(np.abs(LU[k:, k])))
        if LU[r, k] == 0.0:
            raise ValueError(f'B singular: kolom {k} tidak punya pivot tak-nol')
        if r != k:
            LU[[k, r], :] = LU[[r, k], :] # tukar seluruh baris, termasuk multiplier
            lama = perm[k]
            perm[[k, r]] = perm[[r, k]]
        # multiplier l_ik disimpan di tempat elemen yang dieliminasi
        LU[k + 1:, k] /= LU[k, k]
        # baris i dikurangi l_ik * baris k (kolom k+1 ke kanan)
        LU[k + 1:, k + 1:] -= np.outer(LU[k + 1:, k], LU[k, k + 1:])
    if LU[N - 1, N - 1] == 0.0:
        raise ValueError('B singular: pivot terakhir nol')
    return LU, perm

def lu_band(AB: np.ndarray, p: int, q: int) -> tuple[np.ndarray, np.ndarray]:
    AB = AB.copy()
    d = p + q
    N = AB.shape[1]
    piv = np.zeros(N, dtype=int)
    for k in range(N):
        i_maks = min(N - 1, k + p)
        j_maks = min(N - 1, k + p + q)
        r = k + int(np.argmax(np.abs(AB[d:d + i_maks - k + 1, k])))
        if AB[d + r - k, k] == 0.0:
            raise ValueError(f'B singular: kolom {k} tidak punya pivot tak-nol')
        piv[k] = r
        js = np.arange(k, j_maks + 1)
        if r != k:
            sementara = AB[d + k - js, js]
            AB[d + k - js, js] = AB[d + r - js, js]
            AB[d + r - js, js] = sementara
        js = js[1:]
        for i in range(k + 1, i_maks + 1):
            AB[d + i - k, k] /= AB[d, k]
            AB[d + i - js, js] -= AB[d + i - k, k] * AB[d + k - js, js]
    return AB, piv
```

Kode A.2: Solver Gauss berpivot dan QR Givens pada Soal 2.

```
# Extracted verbatim from soal2/report/analyze.py
import numpy as np

def gaussian_pivot(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Solve a square linear system by partial-pivot Gaussian elimination."""
    u = a.astype(float).copy()
    y = b.astype(float).copy()
    n = len(y)
    for k in range(n):
        pivot = k + int(np.argmax(np.abs(u[k:, k])))
        if u[pivot, k] == 0:
            raise ValueError("Singular system")
        if pivot != k:
            u[[k, pivot]] = u[[pivot, k]]
            y[[k, pivot]] = y[[pivot, k]]
        for i in range(k + 1, n):
            factor = u[i, k] / u[k, k]
            u[i, k:] -= factor * u[k, k:]
            y[i] -= factor * y[k]
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (y[i] - np.dot(u[i, i + 1 :], x[i + 1 :])) / u[i, i]
    return x

def givens_least_squares(a: np.ndarray, b: np.ndarray) -> np.ndarray:
    """Apply each Givens rotation to two rows of [A | b], then back-solve."""
    r = a.astype(float).copy()
    d = b.astype(float).copy()
    m, n = r.shape
    for i in range(n):
        for j in range(i + 1, m):
            if r[j, i] == 0:
                continue
            alpha = np.hypot(r[i, i], r[j, i])
            c, s = r[i, i] / alpha, r[j, i] / alpha
            row_i = r[i, i:].copy()
            row_j = r[j, i:].copy()
            r[i, i:] = c * row_i + s * row_j
            r[j, i:] = -s * row_i + c * row_j
            di, dj = d[i], d[j]
            d[i], d[j] = c * di + s * dj, -s * di + c * dj
            r[j, i] = 0.0
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (d[i] - np.dot(r[i, i + 1 :n], x[i + 1 :])) / r[i, i]
    return x
```